



Control Panel Design and Hardware Integration for Lamina Technical Specification Document

Version {1.0}

Akira Alexander Concetti Fukutomi

SW

31/7/2026

Document Control

Status	Reviewing
Development Trigger	SW
Target Version	Lamina 1.8-compatible release
Target Release Date	
Sibling FRD	
Approval Sign-Off	Pending

Revisions

Version	Primary Author	Description of Version	Date Completed	Reviewed By
1.0	Akira Alexander Concetti Fukutomi	Initial draft for Control Panel Handler UI/backend integration design.	31/7 2026	

1. Context

Feature Context

The Control Panel feature provides a centralized Lamina interface for operating and monitoring the AXI Panel Handler hardware, including Rail Width, Conveyor Belt, Outer Barriers, Panel Clamps, and Safety Level Sensors.

The feature resolves the need for a coordinated control layer that synchronizes Motion, Digital IO, Safety Controller, and Panel Handler modules while preventing unsafe hardware operations through software interlocks and state validation.

User commands are translated into backend service requests, which communicate with the native hardware layer. Hardware state changes are continuously monitored and propagated back to the UI to ensure real-time synchronization and safe operation.

2. API & Data Contract

This section defines the backend APIs and service-layer changes introduced or modified to support the Control Panel feature. The APIs provide Panel Handler monitoring, hardware configuration access, axis-state retrieval, position-limit retrieval, and communication with the underlying Motion and native hardware implementations.

Hardware communication is performed through Java service APIs, Remote Interfaces (RMI), JNI wrappers, and native C++ implementations.

2.1 Panel Handler API (Java)

2.1.1 Monitoring Lifecycle

Function	Status	Parameters	Return
startMonitoring()	New	None	void
stopMonitoring()	New	None	void
finishstopMonitoring()	New	ScheduledFuture<?> monitoringTask	void
notifyLimitSensorStatus())	New	None	void
getPanelHandlerPositive LimitStatus()	New	Map<MotionAxisToMoveEnum, Boolean> currentPositiveLimitMap	void
getPanelHandlerNegative LimitStatus()	New	Map<MotionAxisToMoveEnum, Boolean> currentNegativeLimitMap	void
registerPanelHandlerLimitSensorStatusEventListener(listener)	New	PanelHandlerLimitSensorStatusListener	void
unregisterPanelHandlerLimitSensorStatusEventListener(listener)	New	PanelHandlerLimitSensorStatusListener	void

- Supports continuous Panel Handler limit-sensor monitoring during subsystem operation.
- Monitoring is started only when no active monitoring task exists and is not restarted while monitoring shutdown is in progress.
- Panel Handler startup ensures that monitoring is active after successful subsystem startup.
- Monitoring is stopped before Panel Handler shutdown to avoid polling unavailable hardware.

- If a monitoring execution is already in progress, shutdown waits for that execution to complete before clearing the monitoring state and continuing.
- `finishStopMonitoring()` clears the stopped monitoring task and resets the monitoring shutdown state after the active monitoring execution has completed.
- Positive and negative limit-switch states are retrieved for all available Panel Handler axes and published to registered listeners whenever a state change is detected.
- Available axes are obtained through `getAvailableDefinedPanelHandlerAxisList()`.
- Duplicate listener registrations are prevented.
- Panel Handler limit status is polled every 500 ms.

2.1.1.1 [Implementation Logic] `stopMonitoring()`

Panel Handler monitoring uses two synchronization mechanisms because monitoring lifecycle state and active polling execution must be protected separately.

- `synchronized (this)` protects the shared monitoring lifecycle state, including `_monitoringTask` and `_isStoppingMonitoring`. It ensures that monitoring startup and shutdown cannot modify this state concurrently.
- `_monitoringExecutionLock` protects the actual limit-sensor polling execution. The monitoring task holds this lock while accessing the Panel Handler hardware, and `stopMonitoring()` acquires the same lock to wait until any active polling execution has completed.
- `stopMonitoring()` first uses `synchronized (this)` to capture the current monitoring task, mark monitoring shutdown as in progress, and cancel future scheduled executions.
- The `synchronized` block is then released before waiting on `_monitoringExecutionLock`. This prevents the Panel Handler lifecycle monitor from being held while an active hardware poll finishes.
- After the execution lock is acquired, `finishStopMonitoring()` synchronizes on the Panel Handler instance again to safely clear the monitoring-task reference and reset `_isStoppingMonitoring`.
- The task reference is cleared only when it still matches the task being stopped, preventing an older shutdown operation from clearing the state of a different monitoring task.

2.1.2 Panel Handler Limit-Sensor Listener

Function	Status	Parameters	Return
<code>updateAxisPositiveLimit()</code>	New	<code>Map<MotionAxisToMoveEnum, Boolean></code>	void
<code>updateAxisNegativeLimit()</code>	New	<code>Map<MotionAxisToMoveEnum, Boolean></code>	void

- `PanelHandlerLimitSensorStatusListener` was introduced to publish positive and negative limit-state maps from Panel Handler monitoring to registered consumers.
- Each callback provides the latest limit state for the monitored Panel Handler axes.

2.1.3 Panel Handler Operation Listener

Function	Status	Parameters	Return
registerPanelHandlerEventListener()	New	PanelHandlerEventListener	void
unregisterPanelHandlerEventListener()	New	PanelHandlerEventListener	void
panelHandlerOperationBegin()	New	String eventStr	void
panelHandlerOperationEnd()	New	None	void
panelHandlerOnFailure()	New	String failureMessage	void

- Added PanelHandlerEventListener to publish Panel Handler monitoring operation state and monitoring failures.
- Monitoring failures notify registered consumers through panelHandlerOnFailure().
- Monitoring stop initiated by a monitoring failure is wrapped by operation-begin and operation-end notifications.

2.1.3 Home-Offset and Axis-State

Function	Status	Parameters	Return
getHomeOffsetInCounts()	New	MotionAxisToMoveEnum axis	double (counts)
updateHomeOffsetInCounts()	New	MotionAxisToMoveEnum axis, double homeOffset	void
isAxisEnabled()	New	MotionAxisToMoveEnum axis	boolean
getAvailableDefinedPanelHandlerAxisList()	New	None	List<MotionAxisToMoveEnum>

- Retrieves and updates home offsets for Panel Handler axes.
- Home offsets are retrieved from the backend during initialization instead of being hardcoded in the frontend.
- The backend remains the source of truth for stored home-offset values, while the frontend manages editing, unit conversion, and display.
- Exposes the enable state of Panel Handler axes.
- Available Panel Handler axes are retrieved through the Panel Handler-specific backend API instead of relying on the existing generic Motion axis list.

Supported axes include:

- RAIL_WIDTH_INDEPENDENT_AXIS
- LEFT_SAFETY_LEVEL_SENSOR
- RIGHT_SAFETY_LEVEL_SENSOR

2.1.4 Position-Limits

Function	Status	Parameters	Return
getMinimumPositionLimitInNanometers()	New	MotionAxisToMoveEnum axis	double (nm)
getMaximumPositionLimitInNanometers()	New	MotionAxisToMoveEnum axis	double (nm)

- The new position-limit APIs provide configured minimum and maximum limits for supported Panel Handler axes.
 - Position limits are retrieved from the backend instead of using frontend constants.
 - The Panel Handler monitoring system continuously monitors rail-width limit states.
 - When an unsafe limit condition is detected, the Control Panel enters a recovery-required state and prevents further rail movement until reinitialization.
-

2.2 Motion Backend API

2.2.1 Panel Handler Axis Support

The existing Motion backend remains responsible for the generic Motion axes defined by the active Motion configuration.

The Control Panel does not use the generic Motion axis APIs for the following Panel Handler-specific axes:

- RAIL_WIDTH_INDEPENDENT_AXIS
- LEFT_SAFETY_LEVEL_SENSOR
- RIGHT_SAFETY_LEVEL_SENSOR

These axes are accessed through the Panel Handler-specific API instead.

The existing Motion functions, including home-offset retrieval, home-offset update, axis enable-state retrieval, and defined-axis retrieval, continue to operate on the generic Motion-axis set and are not extended by this feature to include the three Panel Handler axes.

2.2.2 Kotlin Manager Lifecycle Integration

Function	Status	Parameters	Return
startUpOrShutDownMotion()	Modified	boolean forceHoming = false	Suspended completion
registerListener()	Modified	None	Unit
unregisterListener()	Modified	None	Unit

startMonitoring()	New	None	Unit
stopMonitoring()	New	None	Unit

- Prevent duplicate listener registration and monitoring startup.
- Start required Safety Controller and Digital IO dependencies only when they are not already started.
- Refresh the configured generic Motion-axis list after successful Motion startup.
- Support forced homing during Rail Width recovery.
- Stop monitoring and unregister listeners before Motion shutdown.

2.3 Digital IO API

2.3.1 Native Access Synchronization

Remote Digital IO operations that access the native Digital IO instance are synchronized to prevent concurrent hardware access.

The synchronized access applies to:

- Digital IO initialization
- Hardware input retrieval
- Digital IO startup
- Digital IO shutdown

A shared native-access lock serializes these operations so that initialization, input reads, startup, and shutdown cannot access the native Digital IO instance concurrently. This prevents hardware polling and lifecycle operations from conflicting while accessing the same native Digital IO resource.

2.3.2 Kotlin Manager Lifecycle Integration

Function	Status	Parameters	Return
startUpOrShutDownDigitalIo()	Modified	boolean startUp	Suspended completion
prepareForHardwareShutdown()	New	None	Suspended completion
startMonitoring()	New	None	void
stopMonitoring()	New	None	Suspended completion

- Updated to support idempotent startup and shutdown.
- Prevents duplicate connections and duplicate monitoring tasks.
- Monitoring is stopped before subsystem shutdown.

- Front-end polling uses coroutine cancellation together with `cancelAndJoin()` to ensure active polling completes before shutdown.

The lifecycle behaviour must satisfy the following requirements:

- Do not reconnect an already connected Digital IO module.
 - Do not initialize or start an already started module.
 - Do not create multiple polling jobs.
 - Stop polling before shutting down Digital IO.
 - Avoid repeated shutdown operations against an already stopped module.
 - Wait for active polling cancellation using `cancelAndJoin()`.
-

2.4 Safety Controller API

2.4.1 Java Monitoring Lifecycle

Function	Status	Parameters	Return
<code>startMonitoring()</code>	Modified	None	void
<code>stopMonitoring()</code>	Modified	None	void

- Updated Safety Controller monitoring to prevent duplicate polling tasks.
- Hardware safety status is polled at the existing Safety Controller monitoring interval.
- Each polling execution is protected by a shared execution lock to prevent monitoring and shutdown from accessing the Safety Controller concurrently.
- Monitoring failures are published to registered listeners and terminate the failed monitoring execution.
- Monitoring is stopped before Safety Controller shutdown.
- Future scheduled executions are cancelled during shutdown, while an execution already in progress is allowed to complete before shutdown continues.

2.4.1.1 [Implementation Logic] `stopMonitoring()`

Safety Controller monitoring uses `_pollingExecutionLock` to synchronize shutdown with an active safety-check execution.

- `cancel(false)` prevents future scheduled executions but does not interrupt a polling execution that is already running.
- Each polling execution holds `_pollingExecutionLock` while `safetyCheck()` is running.
- During `stopMonitoring()`, acquiring `_pollingExecutionLock` therefore blocks until any active safety check has completed.
- No additional state update is required after the lock is acquired because the scheduled-task reference has already been cleared before waiting for the active execution.

- The try block therefore contains only a log statement confirming that polling execution has stopped; the synchronization occurs through acquiring the lock itself rather than through an operation inside the try block.
- The finally block guarantees that `_pollingExecutionLock` is always released.

2.4.2 Kotlin Manager Lifecycle Integration

Function	Status	Parameters	Return
<code>startupOrShutdownSafetyController()</code>	Extended	boolean startUp	void
<code>notifySafetyMonitoringThreadExceptionThrown()</code>	Extended	Exception	void

- Updated to coordinate Safety Controller startup and shutdown with the Control Panel lifecycle.
 - Monitoring failures are published to registered listeners.
 - A monitoring `ErrorCodeException` is propagated after notification so that the failed monitoring execution does not silently continue.
 - Existing machine safety interlocks remain enforced by the backend and hardware.
-

2.5 Remote Interface API

This section defines the new remote methods introduced to expose the Control Panel functionality through the RMI layer.

2.5.1 Home Offset & Axis State

Function	Status	Parameters	Return
<code>remoteGetHomeOffsetInCounts()</code>	New	MotionAxisToMoveEnum axis	double (counts)
<code>remoteUpdateHomeOffsetInCounts()</code>	New	MotionAxisToMoveEnum axis, double homeOffset	void
<code>remoteIsAxisEnabled()</code>	New	MotionAxisToMoveEnum axis	boolean
<code>remoteGetAvailableDefinedPanelHandlerAxisList()</code>	New	None	List<MotionAxisToMoveEnum>

Supports backend retrieval of Panel Handler home offsets, axis enable state, and available Panel Handler axes through the remote service interface.

2.6 Native / JNI API (C++)

This section documents the JNI and native C++ functions introduced to support Panel Handler-specific home-offset and axis-state access.

2.6.1 JNI Interface

Function	Status	Parameters	Return
nativeGetHomeOffsetInCounts()	New	int motionAxisToControl	double
nativeUpdateHomeOffsetInCounts()	New	int motionAxisToControl, double homeOffsetInCounts	void
nativesAxisEnabled()	New	int motionAxisToControl	boolean

- Added JNI support for retrieving and updating home offsets for Panel Handler axes.
- Added JNI support for retrieving the enabled state of Panel Handler axes.
- The selected Panel Handler axis is explicitly specified for each operation.
- Native errors continue to use the existing JNI error-handling mechanism.

2.6.2 Panel Handler Native Implementation

Function	Status	Parameters	Return
PanelHandler::getHomeOffsetInCounts()	New	int motionAxisToControl	double
PanelHandler::updateHomeOffsetInCounts()	New	int motionAxisToControl, double homeOffsetInCounts	void
PanelHandler::isAxisEnabled()	New	int motionAxisToControl	boolean

- Added Panel Handler-specific support for retrieving and updating home-offset values.
- Added support for retrieving the enabled state of Panel Handler axes.
- Added conversion of the supplied axis identifier to the corresponding native MotionAxisEnum.
- Panel Handler-specific axes remain separate from the generic Motion-axis APIs.

2.6.3 Panel Handler Control Implementation

Function	Status	Parameters	Return
PanelHandlerControl::getHomeOffsetInCounts()	New	MotionAxisEnum axisEnum	double

PanelHandlerControl::updateHomeOffsetInCounts()	New	MotionAxisEnum axisEnum, double homeOffsetInCounts	void
---	-----	--	------

- Added native control support for retrieving and updating home-offset values for Panel Handler axes.
- Home-offset operations support the selected Panel Handler axis.

2.7 Data Structures & Types (Kotlin)

The Control Panel frontend exposes immutable Kotlin state objects through StateFlow. This section documents only the data structures introduced or modified for the Control Panel feature.

Hardware-dependent state uses conservative default values before subsystem initialization. Confirmed hardware values replace these defaults after the required modules have successfully started and the initial state has been retrieved. Unless otherwise stated, all state objects are immutable data classes. Updates are performed by creating new instances and publishing them through MutableStateFlow.

2.7.1 PanelConfig

PanelConfig stores the configured panel dimensions and clearance values retrieved for Control Panel validation.

Field	Type
MaximumPanelWidthInNanometers	Double
MinimumPanelWidthInNanometers	Double
MaximumPanelLengthInNanometers	Double
MinimumPanelLengthInNanometers	Double
PanelEdgeClearanceInNanometers	Double

These values are obtained from the existing machine configuration and are not hardcoded in the Control Panel UI.

2.7.2 MachineState

MachineState groups the state of all Control Panel hardware sections into a single immutable object exposed through StateFlow. Updates to individual sections automatically propagate to the UI through reactive state observation.

Field	Type	Default
railWidth	RailWidthModel	RailWidthModel()
belt	BeltModel	BeltModel()
barrier	BarrierModel	BarrierModel()
clamp	ClampModel	ClampModel()
safety	SafetySensorModel	SafetySensorModel()

2.7.3 RailWidthModel

Field	Type	Default
targetWidth	String	"Home"
homeOffset	String	""
disabled	Boolean	true
positiveLimit	Boolean	false
negativeLimit	Boolean	false

The default disabled state prevents Rail Width operations before hardware initialization is complete. Positive and negative limit states are continuously updated by the backend monitoring system and are used to determine whether Rail Width movement remains safe.

2.7.4 BeltModel

Field	Type	Default
running	Boolean	false
directionLTR	Boolean	true

The running and direction values are synchronized with confirmed Panel Handler belt status. Belt state is also used by the Control Panel to prevent unsafe operations such as Rail Width movement or changing conveyor direction while the conveyor is running.

2.7.5 BarrierModel

Field	Type	Default
leftOpen	Boolean	false
rightOpen	Boolean	false

Barrier state is updated from the corresponding Digital IO sensor values.

2.7.6 ClampModel

Clamp state is used by the Control Panel to determine whether conveyor movement is permitted.

Field	Type	Default
clamped	Boolean	false
partiallyClamped	Boolean	false

2.7.7 ClampSensorState

ClampSensorState stores the individual Digital IO clamp inputs used to derive the aggregate clamp status.

Field	Type	Default
fullyClosed	Boolean	false
clampOneClosed	Boolean	false
clampTwoClosed	Boolean	false
clampThreeClosed	Boolean	false
opened	Boolean	false
partiallyClosed	Computed Boolean	Computed

partiallyClosed is a computed property and is not stored independently:

partiallyClosed = !fullyClosed && !opened && (clampOneClosed || clampTwoClosed || clampThreeClosed)

The computed partiallyClosed value avoids storing redundant state and is derived directly from the individual clamp sensor inputs.

2.7.8 SafetySensorModel

The left and right Safety Level Sensor states are maintained independently so that each axis can be moved, monitored, and displayed separately.

Field	Type	Default
leftAxis	AxisState	AxisState()
rightAxis	AxisState	AxisState()

2.7.9 AxisState

Field	Type	Default
position	String	"0.00 mm"
sensorSafe	Boolean	true

homeSafe	Boolean	true
----------	---------	------

The left and right axes maintain independent AxisState objects so that movement, actual position, safety input, and home-sensor state can be updated separately.

2.7.10 RailWidthUiState

RailWidthUiState separates editable operator input from the last applied or confirmed Rail Width state. Separate nanometre values are maintained for backend communication, while user-editable values remain in the currently selected engineering unit.

Field	Type	Default
editingWidth	String	"Home"
editingWidthUnit	MathUtilEnum	NANOMETERS
editingOffset	String	""
editingOffsetUnit	MathUtilEnum	NANOMETERS
editingOffsetActive	Boolean	false
editingWidthActive	Boolean	false
appliedWidth	String	"0"
appliedWidthUnit	MathUtilEnum	NANOMETERS
editingWidthNm	Double	0.0
appliedWidthNm	Double	0.0

The editable and applied values are stored separately so that entering a new value does not immediately overwrite the confirmed hardware state. The applied value is updated only after a successful operation or hardware read-back.

Fields ending with:

- Unit define the engineering unit associated with the corresponding value.
- Active indicate whether the corresponding input is currently in edit mode.
- Nm store the value in nanometres for backend processing and diagram positioning.

2.7.11 SafetyOffsetUiState

SafetyOffsetUiState stores independent offset-editing and actual-position state for the left and right Safety Level Sensor axes.

Field	Type	Default
leftEditingOffset	String	""

leftEditingOffsetUnit	MathUtilEnum	NANOMETERS
leftEditingOffsetActive	Boolean	false
rightEditingOffset	String	""
rightEditingOffsetUnit	MathUtilEnum	NANOMETERS
rightEditingOffsetActive	Boolean	false
leftAppliedPosition	String	SafetyPositionEnum.HOME.displayName
rightAppliedPosition	String	SafetyPositionEnum.HOME.displayName
leftActualPositionInNm	Double	null
rightActualPositionInNm	Double	null

Before subsystem initialization completes, hardware-dependent UI states use default values instead of querying hardware. Actual hardware values are retrieved only after the required subsystems have successfully started, preventing invalid hardware access during application startup.

Because hardware positions are unavailable until initialization and position retrieval complete, the actual-position fields are nullable. A null value must not be interpreted as a physical position of zero.

The applied predefined position and the actual hardware position are stored separately to distinguish the user-requested position from the confirmed hardware position.

2.7.12 SafetyPositionEnum

The Control Panel provides predefined safety-level positions for the V810Qx.

Position	Nominal Position
HOME	Home operation; no fixed non-home target (equivalent to M30L on V810Qx)
M4	10,000,000 nm
M8L	19,000,000 nm
M15L	41,000,000 nm
M22L	67,000,000 nm
M30L	90,000,000 nm

The displayed predefined position is derived from confirmed hardware state. If the corresponding home input is active, the UI displays HOME. Otherwise, the actual hardware position is mapped to the nearest predefined non-Home position.

2.8 State Updates & Notifications

The Control Panel publishes asynchronous hardware state updates through scheduled monitoring tasks, service listeners, and Kotlin StateFlow.

2.8.1 Asynchronous updates

Panel Handler

- Positive limit-state updates
- Negative limit-state updates

Digital IO

- Barrier sensor updates
- Clamp sensor updates
- Safety-level safe-input updates
- Safety-level home-input updates

Safety Controller

- Safety/interlock updates
- Monitoring failure notifications

2.8.2 Explicitly refreshed state

- Rail Width actual position
 - Safety Level Sensor actual positions
 - axis enabled state
 - home-offset values
 - safe-to-continue state
-

2.9 Error Contract

Hardware-related backend operations may throw `ErrorCodeException`. Remote and native failures are converted through the existing error-handling layers. Manager classes wrap applicable backend exceptions as `GeneralException` before exposing them to the ViewModel and UI.

Error Category	Behaviour
Configuration/initialization failure	Operation aborted during hardware initialization
Subsystem not started	Operation rejected until the subsystem has been started
Startup failure	Startup sequence aborted and reported to UI
Hardware timeout	Startup or shutdown operation is aborted if the configured timeout is exceeded
Hardware unavailable	Operation aborted
Invalid axis	Validation failure before hardware access
Safety violation	Operation blocked
Belt running during rail adjustment	Rail width adjustment and homing are rejected
Belt running during clamp operation	Panel Clamp operation blocked
Clamp not open during conveyor start	Conveyor Belt operation blocked
Remote communication failure	RemoteException converted into application exception
Native exception	JNI converts native exception into Java ErrorCodeException
Panel Handler monitoring failure	Listeners notified and Panel Handler monitoring stopped
Safety Controller monitoring failure	Listeners notified and monitoring failure propagated
Polling cancellation during shutdown	Frontend polling completed before subsystem shutdown
Active Panel Handler monitoring during shutdown	Active polling completed before Panel Handler shutdown continues
Shutdown failure	Cleanup attempted before reporting the failure

2.10 Versioning & Compatibility

The Control Panel feature extends the existing Panel Handler API additively without removing existing methods or changing existing method signatures.

New capabilities introduced include:

- Panel Handler home-offset retrieval and update
- Panel Handler axis enable-state query
- Rail Width positive and negative limit monitoring
- Continuous Panel Handler limit monitoring
- Independent Safety Level Sensor control
- Digital IO polling and controlled lifecycle management
- Safety Controller monitoring and controlled lifecycle management

The frontend implementation was aligned with `axi-ic-1.8.3.4.jar`, and `axi-util-1.8.3.4.jar`.

All newly introduced APIs are additive and remain compatible with existing consumers. Existing API methods remain available and existing consumers that do not call the new methods remain functionally compatible. However, the updated frontend, Java API JARs, service implementation, remote interface, JNI bindings, and native DLL must be deployed as a compatible version set when the new Control Panel APIs are used.

3. Consumer System & Modules

This section identifies the software modules that consume or provide services for the Control Panel feature. The ControlPanelViewModel acts as an orchestration layer that coordinates multiple hardware subsystems while exposing a unified interface to the frontend.

3.1 Internal Consumers (within R&D)

The following modules were introduced or modified to implement the Control Panel feature.

Module	Responsibility	Changes Introduced
ControlPanelViewModel	Feature orchestration	Coordinates all subsystem initialization and shutdown, validates user actions, sequences lifecycle operations, manages loading and error states, retrieves confirmed hardware state, and synchronizes it with the frontend
PanelHandlerManager	Panel Handler-specific control and monitoring	Added Rail Width and Safety Level Sensor control, home-offset and axis-state access, configured position limits, Rail Width limit monitoring, dedicated limit-listener handling, and hardware-state synchronization
MotionManager	Motion lifecycle and backend access	Extended Motion lifecycle management, generic Motion-axis retrieval, startup/shutdown sequencing, and monitoring cleanup.
DigitalIOManager	Digital IO lifecycle and status synchronization	Extended lifecycle management, pre-shutdown polling cleanup, duplicate-operation prevention, listener-registration management, synchronized native Digital IO access, and hardware-state synchronization
SafetyControllerManager	Safety lifecycle and interlock monitoring	Extended lifecycle coordination, listener lifecycle handling, monitoring-error processing, interlock-state updates, and startup-state synchronization
ControlPanelUiState	Control Panel state model	Added state models for hardware status, editable offsets, applied values, engineering units, actual positions, and limit states

Module Interaction

The Control Panel does not communicate directly with hardware. Instead, it coordinates subsystem managers responsible for individual hardware domains.

During normal initialization, the ViewModel starts the Panel Handler when required, waits for successful startup, starts the Safety Controller when available, and then retrieves home offsets and the initial Rail Width and Safety Level Sensor states. Dependent Motion and Digital IO lifecycle operations are coordinated by the corresponding subsystem managers, ensuring that subsystem startup, shutdown, and initial hardware-state retrieval occur in the correct sequence.

Hardware state is returned through listener callbacks, scheduled polling, explicit post-operation read-back, and Kotlin StateFlow. The ViewModel combines these updates into the Control Panel UI state.

3.2 External Consumers (via API)

The backend APIs defined by this feature are mainly consumed by ControlPanelScreen, through the ViewModel and managers.

Consumer	Purpose	API Usage
ControlPanelScreen	Primary operator interface for the Panel Handler Control Panel	Observes state exposed by ControlPanelViewModel and invokes ViewModel operations through UI callbacks. It does not access subsystem managers or backend APIs directly

ControlPanelScreen renders the following reusable UI components introduced or modified for the feature:

- SectionCard
- HomeOffsetCell
- RailWidthDiagram
- SafetyLevelDiagram
- BarrierDiagram
- BeltDiagram
- ClampDiagram
- StatusBadge
- LimitStatusBadge
- SegmentedControl

These child components receive state and callbacks from ControlPanelScreen. They do not communicate directly with the ViewModel, subsystem managers, or backend services.

4. Scientific Context & Algorithms

4.1 Units & Standards

The Control Panel uses the following engineering units:

Quantity	Internal Representation	Display
Rail Width	Nanometers (Double)	User-selectable engineering units
Safety Level Sensor Position	Nanometers (Double)	Predefined safety positions or engineering units
Home Offset	Distance value represented as Double	User-selectable engineering units

All calculations performed by the frontend use the existing engineering-unit conversion utilities (MathUtilEnum) to maintain consistency between backend nanometer values, editable input values, and displayed engineering units. Safety Level Sensor positions are internally represented in nanometres and mapped to predefined positions (HOME, M4, M8L, M15L, M22L, and M30L) for display.

5. Performance Requirements

5.1 Computational Performance

The Control Panel performs hardware operations asynchronously using Kotlin coroutines to prevent blocking the user interface. UI state is updated through StateFlow, allowing confirmed hardware-state changes to be reflected automatically.

Monitoring tasks are started only when no active monitoring task exists. Panel Handler monitoring cannot restart while shutdown of the current monitoring task is in progress. Startup and shutdown operations are synchronized and executed sequentially to prevent conflicting lifecycle execution. Active monitoring tasks are cancelled and completed before subsystem shutdown to prevent access to unavailable hardware.

5.2 Throughput & Latency

Component	Update Interval
Digital IO polling	500 ms
Panel Handler limit monitoring	500 ms

- Hardware status is synchronized with the user interface through scheduled polling, listener callbacks, explicit hardware read-back, and StateFlow updates.
- Startup and shutdown operations use timeout-based synchronization to ensure reliable subsystem initialization and prevent indefinite blocking.
- Panel Handler monitoring cancels future scheduled executions and waits for an active polling execution to finish before shutdown continues. Frontend Digital IO polling is cancelled and joined before Digital IO shutdown.

5.3 Resource Constraints

The Control Panel feature runs within the existing Lamina Compose Desktop application and leverages the existing Motion, Panel Handler, Digital IO, and Safety Controller services through the established Java service infrastructure, including RMI, JNI, and the native C++ backend.

The feature extends the existing frontend, Java API, remote interface, JNI, and native implementations without introducing additional hardware, operating system, memory, storage, or network requirements beyond the current platform.

6. Safety & Regulatory

6.1 Safety Classification

The Control Panel is not a safety-critical implementation. It provides operator control and monitoring for the existing Motion, Panel Handler, Digital IO, and Safety Controller subsystems while relying on the existing hardware safety mechanisms and backend safety implementation.

6.2 Safety Interlocks & Fail-Safes

The Control Panel respects the following safety mechanisms:

- All Control Panel operations are permitted only after the Panel Handler, Motion, Digital IO, and Safety Controller modules have successfully started and completed initialization.
- Rail-width adjustment, rail homing, and panel clamp operation are prohibited while the conveyor belt is running.
- The Conveyor Belt can only be started while the Panel Clamps are open.
- The conveyor belt direction cannot be changed while the belt is in motion. The belt must first be stopped before a different direction can be selected.
- Only one hardware operation may be executed at a time. While an operation is in progress, other Control Panel commands are disabled until the current operation completes or is aborted.
- The left and right Safety Level Sensor safe and home inputs are independently mapped and propagated through Digital IO monitoring.
- Non-Home Safety Level Sensor movement operates independently for the left and right axes. The Home operation homes both Safety Level Sensor axes. After movement or homing, the actual hardware position is synchronized with the displayed predefined position.
- The Safety Controller continuously monitors machine interlocks, and detected safety faults are propagated to the user interface.
- Positive and negative limit switches of the rail width axis are continuously monitored. If an unsafe limit condition is detected, the Control Panel displays a reinitialization-required error and prevents further rail-width operation until the system has been reinitialized.
- Hardware startup and shutdown are performed in a controlled sequence, and monitoring tasks are stopped before subsystem shutdown to prevent access to unavailable hardware.

7. Edge Cases & Error Handling

7.1 Out-of-Bounds Parameters

The Control Panel validates hardware operations before execution. Invalid operations are rejected by the backend and reported to the frontend.

Examples include:

- Home-offset operations are validated using the configured Panel Handler axis definitions retrieved from the backend.
- Rail Width movement requests are validated against the configured minimum and maximum position limits before execution.
- Supported Panel Handler axes are validated against the configured backend axis definitions before hardware access.
- Rail Width movement automatically enables the Rail Width axis if it is disabled before issuing the movement command.
- Commands that violate subsystem interlocks or hardware constraints are rejected by the backend and propagated to the frontend as exceptions.

7.2 Hardware Fault Conditions

The feature detects and handles hardware and communication failures through the existing backend exception mechanism.

The implementation handles the following conditions:

- Hardware startup or shutdown failures.
- Motion, Digital IO, Panel Handler, or Safety Controller communication failures.
- Safety Controller monitoring exceptions.
- Digital IO and Panel Handler monitoring failures during polling.
- Panel Handler monitoring failure propagation through the Panel Handler operation listener to the Control Panel.
- Rail Width unsafe limit conditions requiring subsystem reinitialization.
- Hardware service disconnection or unavailable subsystem states.
- Startup or shutdown timeout conditions.
- Concurrent Digital IO initialization, input access, startup, and shutdown are serialized at the remote service layer to prevent simultaneous native Digital IO access.

When a hardware fault occurs, the current operation is aborted and an appropriate exception is propagated from the backend to the frontend. During system shutdown, monitoring tasks are safely terminated to prevent further access to unavailable hardware resources. Where applicable, hardware states are reset to ensure the system can be safely restarted.

For Rail Width recovery, the system performs a controlled subsystem restart. The Panel Handler, Safety Controller, Motion, and Digital IO subsystems are stopped in dependency order, then Digital IO, Motion (with forced homing), Safety Controller, and Panel Handler are restarted. Recovery succeeds only after both Rail Width limits are clear and the current position and axis-enabled state can be retrieved.

8. Technical Backward Compatibility

8.1 Affected Product Variants

The feature affects AXI product variants that use the Panel Handler, Motion, Digital IO, and Safety Controller services.

The updated software supports:

- EtherCAT simulator configuration using cm000.dll.
- Mitsubishi EtherCAT configuration using cm001.dll.
- Kollmorgen motion configurations through MotionManagerKollmorgen.

The feature extends Panel Handler home-offset management, axis enable-state retrieval, Rail Width limit monitoring, Safety Level Sensor control, and subsystem lifecycle management across the supported motion-controller variants. Existing hardware layouts remain unchanged.

8.2 API & Data Format Compatibility

The backend changes are additive and do not remove existing API methods or change existing parameters and return types.

New interfaces introduced by the feature include:

Java Panel Handler API

- `PanelHandler.startMonitoring()`
- `PanelHandler.stopMonitoring()`
- `getAvailableDefinedPanelHandlerAxisList()`
- `getMinimumPositionLimitInNanometers(MotionAxisToMoveEnum)`
- `getMaximumPositionLimitInNanometers(MotionAxisToMoveEnum)`
- `getHomeOffsetInCounts(MotionAxisToMoveEnum)`
- `updateHomeOffsetInCounts(MotionAxisToMoveEnum, double)`
- `isAxisEnabled(MotionAxisToMoveEnum)`
- `registerPanelHandlerLimitSensorStatusEventListener(...)`
- `unregisterPanelHandlerLimitSensorStatusEventListener(...)`
- `registerPanelHandlerEventListener(...)`
- `unregisterPanelHandlerEventListener(...)`

Listener Interfaces

- `updateAxisPositiveLimit(...)`
- `updateAxisNegativeLimit(...)`
- `panelHandlerOperationBegin(...)`
- `panelHandlerOperationEnd()`

- `panelHandlerOnFailure(...)`

Remote Interface

- `remoteGetHomeOffsetInCounts(MotionAxisToMoveEnum)`
- `remoteUpdateHomeOffsetInCounts(MotionAxisToMoveEnum, double)`
- `remoteIsAxisEnabled(MotionAxisToMoveEnum)`
- `remoteGetAvailableDefinedPanelHandlerAxisList()`

JNI Interface

- `nativeGetHomeOffsetInCounts(int)`
- `nativeUpdateHomeOffsetInCounts(int, double)`
- `nativeIsAxisEnabled(int)`
- `nativeGetPanelHandlerAxisList()`

Native C++ Support

- `PanelHandler::getHomeOffsetInCounts(...)`
- `PanelHandler::updateHomeOffsetInCounts(...)`
- `PanelHandler::isAxisEnabled(...)`
- `PanelHandler::getPanelHandlerAxisVector()`
- `PanelHandlerControl::getHomeOffsetInCounts(...)`
- `PanelHandlerControl::updateHomeOffsetInCounts(...)`

The existing Panel Handler APIs for current position, position limits, limit-switch states, rail-width homing, rail-width adjustment, and conveyor operation remain available.

No configuration-file or serialized data format changes are introduced.

Because the new functions extend multiple software layers, the following components must be version-compatible and deployed together:

- Frontend application
- Java API JARs
- Service implementation
- Remote interface and implementation
- JNI bindings
- Native `cm000.dll` or `cm001.dll`

Using an updated frontend or Java API with an older service or native DLL may result in missing-method, interface-version, or native-symbol errors. Existing consumers that do not call the newly added APIs remain functionally compatible.

9. Validation & Reproducibility

The implemented Control Panel functionality was validated primarily through physical-machine testing. Validation focused on correct hardware operation, subsystem lifecycle management, monitoring behaviour, interlock enforcement, and synchronization between backend hardware state and the user interface.

9.1 Reference Configurations

The Control Panel was developed and validated using the following software and hardware configurations.

9.1.1 Development Environment

- Lamina frontend compatible with axi-ic-1.8.3.4.jar
- Lamina frontend compatible with axi-util-1.8.3.4.jar
- Compatible Java service, RMI, JNI, and native implementations
- cm000.dll simulator configuration

9.1.2 Physical Machine

The implemented functionality was validated on an approved AXI machine using:

- cm001.dll
- Panel Handler subsystem
- Motion subsystem
- Digital IO subsystem
- Safety Controller subsystem

9.2 Hardware-in-the-Loop Testing

The implemented Control Panel changes were validated on an AXI physical machine because the simulator does not support full subsystem initialization required for the implemented functionality.

Validation covered the following areas:

- Controlled startup and shutdown of the Panel Handler, Motion, Digital IO, and Safety Controller subsystems.
- Rail Width homing, width adjustment, configured position limits, axis enable behaviour, home-offset retrieval and update, and actual-position read-back.
- Independent non-Home movement of the left and right Safety Level Sensor axes, Home operation of both axes, and synchronization of actual hardware positions with the UI.
- Conveyor Belt, Outer Barrier, and Panel Clamp operations together with their associated software interlocks.

- Rail Width positive and negative limit monitoring, recovery, and reinitialization behaviour.
- Digital IO polling, Panel Handler limit monitoring, and synchronization of monitored hardware states with the frontend.
- Repeated startup and shutdown operations to verify that duplicate monitoring tasks and listener registrations were not created.
- Prevention of Panel Handler monitoring restart while monitoring shutdown is still in progress.
- Completion of active Panel Handler polling before subsystem shutdown continues.
- Correct propagation of left and right Safety Level Sensor safe and home-input states through Digital IO monitoring.
- Prevention of concurrent native Digital IO access during initialization, input reading, startup, and shutdown.
- Monitoring cleanup before subsystem shutdown to prevent polling or monitoring after the corresponding hardware services are disconnected.

Testing was repeated throughout development following backend integration, lifecycle changes, monitoring improvements, and UI-state modifications to verify that the implemented functionality remained synchronized with the physical machine.

- End of Document -